Insurance Claim Frequency Modeling

Statistical and Machine Learning Approach on a Synthetic Portfolio

Jaroslav Drobek

March 23, 2026

Abstract

This text presents a coherent worked example of claim frequency modeling on a synthetically generated portfolio of insured vehicles. The aim is to compare the classical linear model, the Poisson generalized linear model, and a modern tree-based boosting model, including validation, global and local interpretation, and a brief outline of a production pipeline.

Contents

The aim of this study is to compare classical statistical models and modern machine learning methods in modeling the frequency of insurance claims on a synthetic portfolio. Special emphasis is placed on the balance between predictive accuracy, interpretability, and practical applicability.

1 Problem statement

Consider a portfolio of clients for whom we have the following variables available:

- driver's age (`age`),
- length of driving experience (`experience`),
- vehicle power (`car_power`),
- annual mileage (`mileage`),
- bonus–malus class (`bonus_malus`).

The target variable is the number of insurance claims

$$Y \in \{0, 1, 2, \dots\}.$$

The task is to construct a claim frequency model and assess:

1. the suitability of the individual model classes,
2. predictive quality,
3. interpretability of the results,
4. potential business applicability.

2 Theoretical background

2.1 Linear model

The classical linear model assumes the relationship

$$Y = X\beta + \varepsilon.$$

For count data, this model is problematic because it also permits negative predictions and is not naturally tied to the discrete nature of the explained variable.

2.2 Poisson GLM

A natural model for frequency data is the Poisson generalized linear model:

$$Y \sim \text{Poisson}(\lambda), \qquad \log(\lambda) = X\beta.$$

This model:

- respects the non-negativity of the mean,
- provides interpretable coefficients,
- forms the standard foundation of actuarial practice.

2.3 Boosting model

The gradient boosting tree model makes it possible to capture:

- nonlinear relationships,
- interactions between variables,
- threshold and segment effects.

At the cost of higher complexity, it may offer better predictive performance.

3 Generation of synthetic data

A synthetic dataset of size $n = 5000$ observations was created. The generative mechanism was chosen so that it contains nonlinearities and realistic risk variability.

3.1 Data structure

Variables used:

Variable	Meaning
`age`	driver's age
`experience`	length of driving experience
`car_power`	vehicle power
`mileage`	annual mileage
`bonus_malus`	tariff class
`claims`	number of claims

3.2 Data sample

The data were synthetically generated so as to realistically correspond to a typical motor insurance portfolio. The use of synthetic data allows full control over the risk structure while simultaneously eliminating limitations associated with the protection of personal data.

Table 1: Sample of synthetically generated data

age	experience	car$_p ower$	mileage	bonus$_m alus$	claims
56	32	92.810610	20230.566569	4	0
69	42	137.871259	11872.708437	3	2
46	19	66.422355	22919.942917	0	3
32	11	194.875500	19872.900523	4	3
60	35	76.217748	13120.602744	4	4
25	2	147.204548	5634.287201	2	4
78	57	86.818169	23753.757032	3	2
38	17	157.634207	10368.841646	4	1
56	37	166.523205	25861.843587	4	5
75	56	151.012959	15507.256501	4	0

3.3 Generative scheme

Claim intensity was modeled in the form

$$\lambda = \exp(f(x)),$$

where $f(x)$ is a nonlinear function of the predictors. The number of claims itself was then generated from a Poisson distribution.

4 Exploratory analysis

In the first phase, the following were examined:

- basic descriptive statistics,
- distribution of the target variable,
- relationship between the mean and the variance,
- simple graphical dependencies.

Remark 4.1. For count data, it is crucial to verify whether the distribution is strongly asymmetric with a predominance of zero values, which is typical for frequency tasks.

5 Models

5.1 OLS as baseline

The linear model was used as a reference baseline. Its role is primarily didactic: it shows why count data require a more suitable model class.

Here it is appropriate to insert a brief coefficient table or a shortened commentary on the OLS results.

5.2 Poisson GLM

The Poisson GLM was estimated in the form

$$\log(\lambda) = \beta_0 + \beta_1\,\texttt{age} + \beta_2\,\texttt{mileage} + \beta_3\,\texttt{car\_power} + \beta_4\,\texttt{bonus\_malus} + \beta_5\,\texttt{experience}.$$

After exponentiating the coefficients,

$$\exp(\beta_j)$$

the parameters can be interpreted as multiplicative changes in the expected frequency.

5.3 XGBoost

The boosting model was trained with a Poisson objective. This produced a model capable of capturing nonlinear and interaction effects, which the GLM can only capture to a limited extent without further extension.

6 Model comparison

The individual models were compared in terms of predictive accuracy and practical applicability. In addition to the absolute error (MAE), we also monitored the proportions of exact matches after rounding the prediction and the proportions of cases in which the prediction differed from the actual value by at most one claim.

Table 2: Summary of basic model metrics

Metric	Value
Mean claims	1.9646
Variance claims	2.5935
Baseline MAE (full sample)	1.2350
OLS MAE	1.0976
GLM Poisson MAE	1.0858
XGBoost MAE	1.0570
OLS exact match	0.2796
GLM exact match	0.2844
XGB exact match	0.2920
OLS within ±1	0.7334
GLM within ±1	0.7474
XGB within ±1	0.7618
Baseline MAE (test)	1.2133
XGB train MAE	1.0553
XGB test MAE	1.0740
XGB test/train ratio	1.0177

The average number of claims per client is approximately 1.96 with a variance of 2.59, which corresponds to a medium-risk portfolio without an extreme dominance of zero observations. In such a context, the absolute model error is naturally on the order of one insurance claim.

A trivial baseline model predicting the constant mean achieves an MAE of approximately 1.24. All considered models clearly outperform this baseline, which confirms that they make use of the information contained in the explanatory variables.

The linear OLS model provides an improvement over the baseline, but its use for count data is methodologically limited. The Poisson GLM represents a statistically more natural approach and shows slightly better results, while maintaining high interpretability of the parameters.

The lowest error is achieved by the XGBoost model, which is able to capture nonlinear dependencies and interactions between variables. The results for exact matches and matches within a tolerance of ± 1 also indicate its slight advantage over the classical models.

From a practical point of view, it is important that the differences between GLM and XGBoost are not dramatic. This corresponds to experience from applied practice, where GLM often provides a strong and transparent baseline, while machine learning models function as a more powerful, though more complex, alternative.

The results suggest that modern machine learning methods may provide some improvement in predictive accuracy, however, the classical GLM remains a strong and transparent reference approach.

7 Validation

7.1 Train/test split

The data were divided into training and test parts in a 70:30 ratio. The model was trained only on the training set and subsequently evaluated on an independent test set, which makes it possible to assess its true generalization ability.

7.2 Baseline comparison

The performance of the model was compared with a trivial baseline predicting a constant value equal to the average number of claims in the data. The required minimum is to outperform this baseline.

The XGBoost model achieves a lower absolute error on the test set than the baseline, which confirms that it makes use of the information contained in the explanatory variables and provides genuinely informative predictions.

7.3 Overfitting check

The ratio of test and training error was approximately

$$\frac{\mathrm{MAE}_{test}}{\mathrm{MAE}_{train}} \approx 1.02.$$

Such a low value indicates very good generalization and the absence of significant overfitting. The model maintains a similar level of accuracy on both known and previously unseen data.

7.4 Calibration

Calibration analysis was performed by deciles of predicted risk. The points of the calibration curve lay close to the diagonal, which indicates that the model is on average well aligned with the actual number of claims and correctly reproduces the relative level of risk across the portfolio.

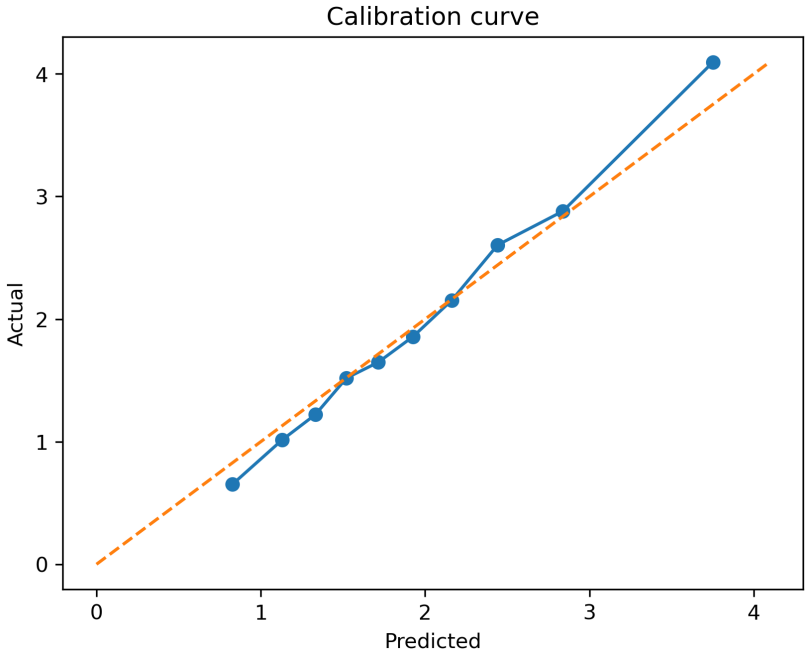

Figure 1: Model calibration curve

8 Model interpretation

8.1 Global interpretation

SHAP analysis provides local explanations of individual predictions, which complement the global view given by the parameters of linear models.

Global interpretation corresponds to the question:

> How does the model work at the level of the entire population?

Tools used:

- feature importance,
- partial dependence plots,
- SHAP summary plot.

8.2 Partial dependence

Partial dependence plots showed:

- an almost monotonic increase in risk with mileage,
- nonlinear behavior for vehicle power,
- a weaker and locally variable effect of age.

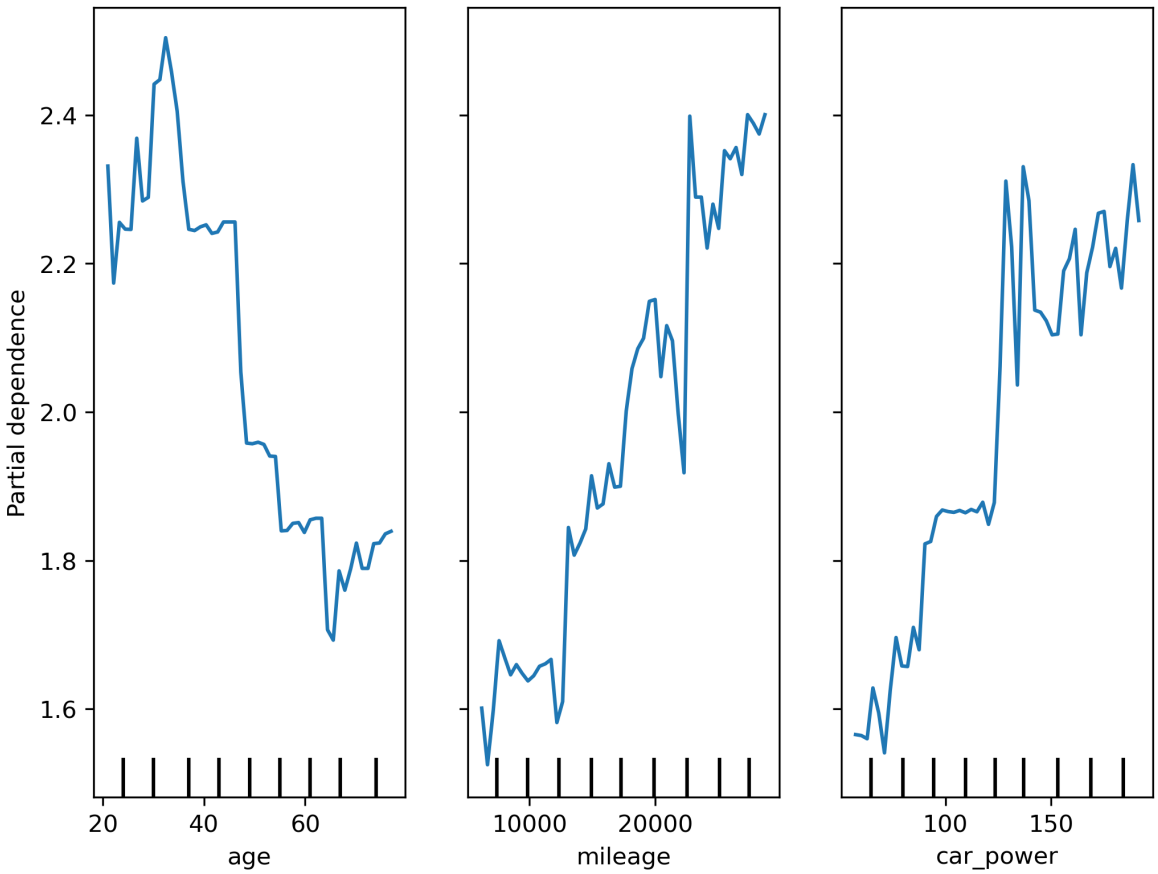

Figure 2: Partial dependence plots of selected variables

8.3 SHAP: global interpretation

The SHAP summary plot made it possible to rank variables by their importance and at the same time observe:

- direction of influence,
- nonlinearities,
- interactions.

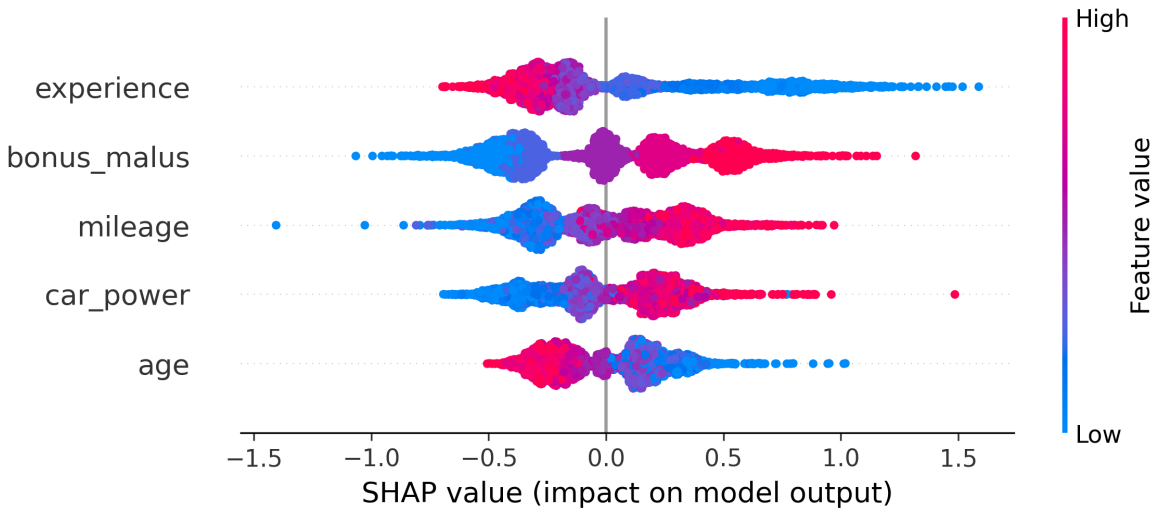

Figure 3: Global SHAP interpretation

8.4 Local interpretation

Local interpretation corresponds to the question:

> Why did the model make exactly this decision for a particular client?

For this purpose, the following were used:

- SHAP waterfall plot,
- client underwriting profile.

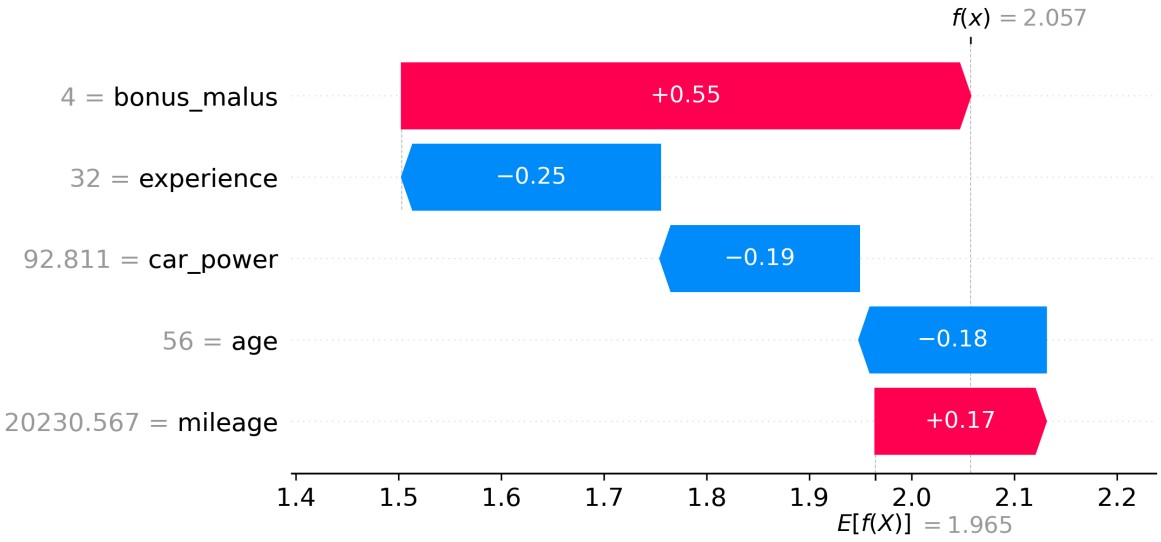

Figure 4: Local SHAP interpretation for a selected client

9 Underwriting profile

On the basis of local SHAP values, a brief underwriting comment can be created.

Example 9.1. The selected client has a predicted claim frequency of 0.200 compared to the portfolio average of 0.171, i.e. approximately 1.17× higher risk. The main factors increasing the risk are vehicle power and bonus–malus, while mileage acts in this case in a risk-reducing direction.

10 Production pipeline

The notebook represents the research and exploratory layer. In a production environment, the model would be divided into:

- training script,
- storage of the model artifact,
- separate scoring script,
- validation and monitoring.

Schematically:

$$\text{data} \to \text{preprocessing} \to \text{model} \to \text{validation} \to \text{scoring} \to \text{report}.$$

11 Business applicability

The model is business-applicable only if:

1. it is more accurate than simple baselines,
2. it generalizes to new data,
3. it is sufficiently interpretable,
4. it leads to an actionable decision.

Possible areas of application:

- pricing,
- portfolio segmentation,
- underwriting,
- identification of risk groups.

At the same time, it holds that in a regulated environment the GLM may still have the advantage of higher transparency, while the ML model plays the role of a challenger or a supplementary tool.

12 Conclusion

The presented example shows the entire life cycle of an analytical task:

problem statement → data → EDA → model → validation → interpretation → pipeline → business conclusion.

The Poisson GLM provides a natural and well-interpretable foundation for claim frequency modeling. XGBoost, by contrast, makes it possible to capture nonlinear and interaction structures, and thereby potentially increase predictive performance. The combination of both approaches — statistical and machine-learning — has proven to be exceptionally strong in applied practice.

The study shows that in claim frequency modeling, machine learning methods may offer higher predictive accuracy, but at the cost of lower interpretability. The Poisson GLM remains a robust and transparent tool, while models of the XGBoost type represent a suitable complement for tasks where pure predictive performance is the priority.

A Appendix: data generation

Here it is appropriate to insert a shortened Python code snippet for data generation.

Listing 1: Skeleton of synthetic data generation

```
np.random.seed(42)
n = 5000

age = np.random.randint(18, 80, n)
experience = np.clip(age - np.random.randint(18, 30, n), 0, None)
car_power = np.random.uniform(50, 200, n)
mileage = np.random.uniform(5000, 30000, n)
bonus_malus = np.random.randint(0, 5, n)

risk = (
    0.02 * (age - 40) ** 2 / 100
    + 0.00002 * mileage
    + 0.003 * car_power
    - 0.02 * experience
    + 0.15 * bonus_malus
)

claims = np.random.poisson(np.exp(risk))

df = pd.DataFrame({
    "age": age,
    "experience": experience,
    "car_power": car_power,
    "mileage": mileage,
    "bonus_malus": bonus_malus,
    "claims": claims
})

X = df.drop(columns="claims")
y = df["claims"]
```

B Appendix: model training

Listing 2: Skeleton of model training

```
model = XGBRegressor(
    n_estimators=200,
    max_depth=4,
    learning_rate=0.05,
    random_state=42
)
model.fit(X, y)
```

C Appendix: scoring

Listing 3: Skeleton of scoring

```
import joblib
import pandas as pd

model = joblib.load("models/xgb_frequency_model.joblib")

new_client = pd.DataFrame([{
    "age": 45,
    "experience": 20,
    "car_power": 120,
    "mileage": 15000,
    "bonus_malus": 1
}])

prediction = model.predict(new_client)

print("Predicted number of claims:", prediction[0])
```

D Appendix: interpretation

Listing 4: Skeleton of interpretation

```
model = joblib.load("models/xgb_frequency_model.joblib")

pred = model.predict(X)

# CALIBRATION
bins = pd.qcut(pred, 10, duplicates="drop")
cal = pd.DataFrame({"pred": pred, "actual": y, "bin": bins})
cal = cal.groupby("bin").mean()

plt.figure()
plt.plot(cal["pred"], cal["actual"], "o-")
plt.plot([0, cal.values.max()], [0, cal.values.max()], "--")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Calibration curve")

plt.savefig(f"{FIG_DIR}/calibration.png",
            dpi=300, bbox_inches="tight")
plt.close()

# PDP
features = ["age", "mileage", "car_power"]
```

```
fig, ax = plt.subplots(figsize=(8, 6))
PartialDependenceDisplay.from_estimator(
    model, X, features, grid_resolution=50, ax=ax
)

plt.savefig(f"{FIG_DIR}/pdp.png",
            dpi=300, bbox_inches="tight")
plt.close()

# SHAP
explainer = shap.Explainer(model)
shap_values = explainer(X)

# summary plot
shap.summary_plot(shap_values, X, show=False)
plt.savefig(f"{FIG_DIR}/shap_summary.png",
            dpi=300, bbox_inches="tight")
plt.close()

# waterfall for the first client
shap.plots.waterfall(shap_values[0], show=False)
plt.savefig(f"{FIG_DIR}/shap_waterfall_client0.png",
            dpi=300, bbox_inches="tight")
plt.close()
```

E Appendix: metrics table

Listing 5: Skeleton of scoring

```
# Datasets for the models
X_ml = df[["age", "mileage", "car_power", "bonus_malus", "experience"
    ]].astype(float)
y = df["claims"]
X_glm = sm.add_constant(X_ml)

# Baseline
mean_claims = y.mean()
var_claims = y.var()

baseline_pred = np.full(len(y), mean_claims)
baseline_mae = mean_absolute_error(y, baseline_pred)

# OLS
ols_model = sm.OLS(y, X_glm).fit()
pred_ols = ols_model.predict(X_glm)
```

```
# GLM Poisson
glm_model = sm.GLM(y, X_glm, family=sm.families.Poisson()).fit()
pred_glm = glm_model.predict(X_glm)

# XGB full-sample (for model comparison)
xgb_full = XGBRegressor(
    objective="count:poisson",
    n_estimators=200,
    max_depth=3,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)
xgb_full.fit(X_ml, y)
pred_xgb = xgb_full.predict(X_ml)

# Rounded predictions
rounded_ols = np.rint(np.clip(pred_ols, 0, None)).astype(int)
rounded_glm = np.rint(np.clip(pred_glm, 0, None)).astype(int)
rounded_xgb = np.rint(np.clip(pred_xgb, 0, None)).astype(int)

# In-sample metrics
mae_ols = mean_absolute_error(y, pred_ols)
mae_glm = mean_absolute_error(y, pred_glm)
mae_xgb = mean_absolute_error(y, pred_xgb)

exact_ols = (rounded_ols == y).mean()
exact_glm = (rounded_glm == y).mean()
exact_xgb = (rounded_xgb == y).mean()

within1_ols = (np.abs(rounded_ols - y) <= 1).mean()
within1_glm = (np.abs(rounded_glm - y) <= 1).mean()
within1_xgb = (np.abs(rounded_xgb - y) <= 1).mean()

# Train/test validation for XGB
X_train, X_test, y_train, y_test = train_test_split(
    X_ml, y, test_size=0.3, random_state=42
)

xgb_tt = XGBRegressor(
    objective="count:poisson",
    n_estimators=200,
    max_depth=3,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
```

```
    random_state=42
)
xgb_tt.fit(X_train, y_train)

pred_train = xgb_tt.predict(X_train)
pred_test = xgb_tt.predict(X_test)

mae_train = mean_absolute_error(y_train, pred_train)
mae_test = mean_absolute_error(y_test, pred_test)
test_train_ratio = mae_test / mae_train

baseline_test_pred = np.full(len(y_test), y_train.mean())
baseline_test_mae = mean_absolute_error(y_test, baseline_test_pred)

# Metrics table
metrics = pd.DataFrame([
    ("Mean claims", mean_claims),
    ("Variance claims", var_claims),
    ("Baseline MAE (full sample)", baseline_mae),
    ("OLS MAE", mae_ols),
    ("GLM Poisson MAE", mae_glm),
    ("XGBoost MAE", mae_xgb),
    ("OLS exact match", exact_ols),
    ("GLM exact match", exact_glm),
    ("XGB exact match", exact_xgb),
    ("OLS within  1 ", within1_ols),
    ("GLM within  1 ", within1_glm),
    ("XGB within  1 ", within1_xgb),
    ("Baseline MAE (test)", baseline_test_mae),
    ("XGB train MAE", mae_train),
    ("XGB test MAE", mae_test),
    ("XGB test/train ratio", test_train_ratio),
], columns=["Metric", "Value"])

metrics.to_latex(
    os.path.join(TAB_DIR, "metrics_summary.tex"),
    index=False,
    float_format="%.4f",
    escape=False
)
```